

IBM z/OS Debugger Host Installation Automation

User Guide

Developed By:

**Edison Kwok
Sindhu Prakash
Pavithra Velusamy**

Document Version: 1.0

Last Updated: 2026-06-11

Automation Version: Compatible with z/OS Debugger
VHR0M0

© Copyright IBM Corporation

Table of Contents

Overview	3
Purpose	3
What Gets Installed	3
Key Benefits	3
Prerequisites.....	3
Required Files	3
Software Requirements	5
z/OS System Requirements	7
Network Requirements.....	7
Access Requirements.....	8
Installation Checklist	8
Installation Procedures.....	8
Pre-Installation Setup.....	9
Installation – Automation Execution	13
Specific Use Case Scenarios.....	17
Scenario 1: Recovery - Reinstallation	17
Scenario 2: Upgrade - Applying New PTF	18
Scenario 3: Multi-Region CICS Setup	19
Scenario 4: Rollback to Previous Version	19
Post-Installation Verification	20
Verification Checklist	20
Functional Testing	21
Performance Verification	22
Troubleshooting Failed Verification	22
Troubleshooting	23
Common Issues and Solutions	23
Diagnostic Commands	27
Getting Help	28
Appendices	29
Appendix A: Variable Reference Table	29
Appendix B: Port Reference (With respect to H3N or equivalent systems).....	30

Appendix C: Dataset Naming Conventions	30
Appendix D: JCL Templates	31
Appendix E: Security Resources	31
Appendix F: File Structure.....	32
Appendix G: Related Documentation.....	33
Appendix H: Quick Reference Commands.....	33
Conclusion.....	33

Overview

Purpose

This automation framework provides a complete, end-to-end solution for installing and configuring IBM z/OS Debugger on z/OS systems using Ansible. The automation eliminates manual configuration steps, reduces installation time, and ensures consistent deployments across environments.

What Gets Installed

The automation installs and configures the following components:

- **IBM z/OS Debugger Base Product** - Core debugging functionality
- **SMP/E Installation** - Software installation and maintenance
- **PTF (Program Temporary Fix)** - Latest updates and fixes
- **RDS (Remote Debug Server)** - Remote debugging capabilities
- **DPS (Debug Profile Server)** - Centralized debug profile management
- **CICS Debugging Support** - Debug CICS applications
- **IMS Debugging Support** - Debug IMS applications (MPP, BMP)
- **SSL/TLS Security** - Secure debug connections

Key Benefits

- **Automated Installation** - Reduces installation time from hours to minutes
- **Consistent Configuration** - Eliminates human error in configuration
- **Repeatable Process** - Easy to replicate across multiple environments
- **Version Control** - All configurations tracked in Git
- **Rollback Capability** - Quick cleanup and reinstallation
- **Documentation** - Self-documenting through Ansible playbooks

Prerequisites

Required Files

You need to download and place the following files in this directory:

1. Base Product RELFILES

- **Filename:** HADRXXX.pax.Z
- **Description:** IBM z/OS Debugger base product installation package (RELFILES)
- **Required for:** SMP/E installation (Step 2)

2. Latest PTF Package

- **Filename:** IBM.HADRXXX.PTF####(or latest available PTF)
- **Description:** Program Temporary Fix (PTF) package for the debugger
- **Required for:** PTF installation (Step 3)

Download Instructions

1. **Obtain the files** from one of the following sources:
 - IBM Shopz (<https://www.ibm.com/software/shopzseries/>)
 - IBM Fix Central (<https://www.ibm.com/support/fixcentral/>)
 - Your IBM representative or support contact
2. **Download both files** to your local machine
3. **Place the files** in this files/ directory:

```
Debug-Config-Automation/  
└─ files/  
    ├── README.md  
    ├── HADRXXX.pax.Z  
    └─ IBM.HADRXXX.PTF####
```

4. **Verify the files** are in place:

```
ls -lh files/
```

You should see both files listed with their respective sizes.

Update Configuration

After placing the files in this directory, you **MUST** update the variables.yml file in the root directory to match your file names:

- Edit variables.yml

```
# --- PTF Installation Variables ---  
ptf_file: IBM.HADRXXX.PTF#### # Update this if your PTF filename is different  
SMP_file: HADRXXX.pax.Z      # Update this if your RELFILES filename is different
```

Important: If you download a newer PTF version, update the ptf_file variable to match the exact filename.

File Naming Convention

- **RELFILES:** Typically named HADRXXX.pax.Z (where HADRXXX is the FMID for the debugger host)
- **PTF Files:** Named in format IBM.HADRXXX.PTF#### (where PTF#### is the actual PTF number)

Verification Checklist

Before running the automation, verify:

- Both required files are present in the files/ directory
- File names in variables.yml match the actual file names
- Files are not corrupted (check file sizes match expected values)
- You have read access to both files

Software Requirements

Software	Minimum Version	Purpose	Installed On
Ansible	2.18.7 or higher	Automation engine	Control Node
Ansible IBM z/OS Core	1.14.1 or higher	z/OS-specific modules	Control node
Python	3.12 or higher	Ansible runtime	Control node
z/OS	V2.4 or higher	Target operating system	z/OS system
ZOAU	Latest	z/OS Open Automation Utilities	z/OS system

Note:

1. For supported Ansible and Python version combinations, see [Compatibility Breakdown > Ansible Python Compatibility](#).
2. For supported IBM z/OS Core Collection, z/OS, and ZOAU version combinations, see [Compatibility Breakdown > IBM z/OS Core Collection Requirements](#).

3. Control node - The workstation, laptop, or server from which Ansible is executed. Ansible, Python, and the Ansible IBM z/OS Core Collection are installed on this system.
4. z/OS system - The target z/OS environment managed by Ansible. z/OS and ZOAU must be available on this system.

Compatibility Breakdown

- Ansible Python compatibility

Ansible Version [1, 2] Supported Python (Control Node)

Ansible 2.20+	Python 3.12 - 3.14
Ansible 2.19	Python 3.11 - 3.13
Ansible 2.18	Python 3.11 - 3.13

- For optimal compatibility with the *ibm.ibm_zos_core* collection, Python version 3.12 is highly recommended (though 3.9+ is generally supported).
- The following table summarizes the requirements for recent versions of the [IBM z/OS core collection](#):

Collection Version	Controller (ansible-core)	Managed Node (z/OS)	ZOAU Version	Python (z/OS)
1.14.x - 1.15.x	>=2.15.x	V2R5 - V3R1	1.3.4 - 1.3.5	3.12.x - 3.13.x
1.12.x - 1.13.x	>=2.15.x	V2R5 - V3R1	1.3.2 - 1.3.3	3.9 - 3.11
1.11.x	>=2.15.x	V2R4 - V2Rx	>=1.3.1, < 1.4.0	3.9 - 3.11
1.9.x	<=2.14.x	V2R4+	>=1.2.5, < 1.3.0	3.9 - 3.12

z/OS System Requirements

Required Products (if configuring)

- **IBM Java** - For DPS server (required)
- **WebSphere Liberty** - For DPS server (required)
- **CICS Transaction Server** - For CICS debugging (optional)
- **IMS** - For IMS debugging (optional)

Storage Requirements

- **DASD Space:** Minimum 500 cylinders on 3390 volumes
- **zFS Space:** Minimum 500 MB for debugger files
- **Temporary Space:** 200 MB for installation files

System Resources

- **APF Authorization:** Required for SVC installation
- **RACF/SAF:** Required for security definitions
- **TCP/IP:** Required for debug server connectivity

Network Requirements

Port	Purpose	Direction
22 (or custom)	SSH connectivity	Control → z/OS
###00	Liberty server	Client → z/OS
###01	CICS DTCN port	Client → z/OS
###02	CICS secure port	Client → z/OS
###03	IMS debugger port	Client → z/OS
###04	IMS DTCN port	Client → z/OS

Note:

Client → z/OS

- Source: Developer workstation/IDE (IBM Developer for z/OS, VS Code, Eclipse)
- Destination: z/OS debug servers
- Purpose: Remote debugging sessions
- Ports: Multiple debug service ports (###00-###04)
- Protocol: TCP/HTTPS
- Usage: Actual debugging activities after installation

Control → z/OS

- Source: Ansible control node (your local machine/workstation)
- Destination: z/OS mainframe system

- Purpose: Ansible automation and SSH connectivity
- Port: 22 (or custom SSH port)
- Protocol: SSH
- Usage: Running Ansible playbooks, executing commands, managing z/OS system during installation

Access Requirements

User Permissions

- **SSH Access:** Passwordless SSH to z/OS system
- **TSO Access:** Ability to submit jobs and run commands
- **Dataset Access:** CREATE, READ, WRITE, DELETE permissions
- **RACF Permissions:** Authority to define security resources
- **Root Access:** Optional, for certain configurations (via surrogate)

Required Datasets Access

- **SYS1.PARMLIB:** For product registration
- **SYS1.PROCLIB** or custom PROCLIB: For started tasks
- **APF-authorized libraries:** For SVC installation

Installation Checklist

Before starting the installation, verify:

- Ansible and required collections installed
- SSH connectivity to z/OS established (passwordless)
- Required files downloaded and placed in `files/` directory
- DASD volumes available with sufficient space
- Network ports available and not in use
- User has necessary permissions
- CICS/IMS systems available (if configuring)
- Java and Liberty installed on z/OS
- Backup of existing configuration (if upgrading)

Installation Procedures

This Pre-Installation Setup section covers the installation and verification of Ansible and the Ansible IBM z/OS Core collection. Python, z/OS, and ZOAU are expected to be pre-

installed and available in the environment. Verify that they meet the minimum version requirements listed in the Prerequisites section before proceeding

Pre-Installation Setup

Step 1: Install Ansible and Dependencies

On your control node (laptop/workstation):

```
# Install Ansible (Check Compatibility breakdown under Prerequisites)
pip install ansible (installs latest version)
```

or

```
pip install ansible-core x.x.x (user defined version)
```

```
# Install IBM z/OS Core collection (Check Compatibility breakdown under Prerequisites)
ansible-galaxy collection install ibm.ibm_zos_core
```

Note: We may get warnings on the compatibility. This can be ignored or it can be suppressed by doing a force install using the following command

```
ansible-galaxy collection install ibm.ibm_zos_core -force
```

```
# Verify installation
ansible --version
ansible-galaxy collection list | grep ibm.ibm_zos_core
```

Expected Output:

```
ansible [core 2.18.7]
ibm.ibm_zos_core      1.14.1
```

Step 2: Setup SSH Connectivity

If there is already an existing SSH key on our Local machine, the same can be re used and no need to generate a new SSH key.

```
# Generate SSH key (if needed)
ssh-keygen -t rsa -b 4096
```

```
# Copy public key to z/OS
ssh-copy-id -p ###22 userid@hostaddress
```

```
# Test connection
ssh -p ###22 userid@hostaddress
```

Step 3: Clone Repository and Configure

```
# Clone repository
```

```
git clone https://github.com/IBM/z_ansible_collections_samples.git
```

```
cd z_ansible_collections_samples/z_system_automation/zos_debugger_install_config_automation
```

Step 4: Edit Configuration files

- `vi variables.yml`

This file contains all the variables that needs to be updated based on the host environment where this debugger will be setup by the system Admin/ System Programmer)

- `vi inventories/inventory.yml`

This file is required for Connection establishment)

- `vi inventories/host_vars/zos_host.yml`

This file has the information on paths to Ansible, Python and also has option for encrypting passwords using Ansible Vault)

Configuration Guide

1. variables.yml - Main Configuration File

This is the primary configuration file where you define all installation parameters.

SMP/E Installation Variables

```
# SMP/E Installation Configuration
```

```
smphlq: IBMUSER.HADRH00      # High-Level qualifier for SMP/E datasets
```

```
volser: T#####             # Volume for SMP/E CSI and control data sets
```

```
tvol: T#####               # Target volume for installed libraries
```

```
dvol: T#####               # Distribution volume for RELFILES
```

```
eqa_hlq: EQAW.VHR0M0         # High-Level qualifier for debugger data sets
```

```
eqa_path_prefix: "/u/ibmuser/" # Base path for debugger files (must end with /)
```

Parameter Details:

Parameter	Description	Example	Notes
smphlq	HLQ for SMP/E datasets	IBMUSER.HADRH00	Used for CSI, SMPLOG, etc.
volser	Volume for SMP/E control	T#####	Must have 100+ cylinders free
tvoll	Target library volume	T#####	Must have 200+ cylinders free
dvol	Distribution volume	T#####	Must have 150+ cylinders free
eqa_hlq	Debugger dataset HLQ	EQAW.VHR0M0	All debugger datasets use this
eqa_path_prefix	zFS base path	/u/ibmuser/	Must end with /

Complete variables.yml Reference

```
# =====
#
# IBM z/OS Debugger Automation - Configuration Variables
# =====
#

# --- SMP/E Installation Variables ---
smphlq: IBMUSER.HADRH00      # SMP/E dataset high-level qualifier
volser: T#####             # SMP/E control dataset volume
tvoll: T#####              # Target Library volume
dvol: T#####               # Distribution Library volume
eqa_hlq: EQAW.VHR0M0        # Debugger dataset HLQ
eqa_path_prefix: "/u/ibmuser/" # zFS base path (must end with /)

# --- PTF Installation Variables ---
remote_path_relfles: "/u/ibmuser/SMP/relfiles" # Temporary RELFILES path
ptf_file: IBM.HADRXXX.PTF#####              # PTF filename

# --- Product Configuration Variables ---
PARMLIB: USER.PARMLIB      # PARMLIB dataset
IFAPRDxx: TZ               # IFAPRD member suffix
registration: EQAWIFAT     # Registration template member

# --- IVP Variables ---
ivp_LNGPRFX: IGY.V6R4M0    # COBOL compiler HLQ
```

```

ivp_LIBPRFX: CEE                                # Language Environment HLQ

# --- DPS Configuration Variables ---
PROCLIB: USER.PROCLIB                          # Started task PROCLIB
DFHHLQ: DFH.V6R3M0.CICS                        # CICS installation HLQ
CFGDIR: /u/ibmuser/etc/debug                   # Configuration directory
WRKDIR: /u/ibmuser/var/debug                   # Working directory
JAVADIR: /usr/lpp/java/J17.0_64                # Java installation path
liberty_dir: /usr/lpp/liberty_zos/current      # Liberty installation path
liberty_port: ###00                           # Liberty server port

# --- CICS Configuration Variables ---
CSDPRFX: CICS61.CICS01                        # CICS CSD dataset prefix
dtcn_port: ###01                             # DTCN transaction port
cics_region: CICS01                          # CICS region name
port_internal: ###01                         # Internal debug port
port_external_secure: ###02                  # External secure debug port

# --- IMS Configuration Variables ---
ims_region: IMSREG1                          # IMS region name
ims_debugger_port: ###03                     # IMS debug server port
ims_dtcn_port: ###04                        # IMS DTCN port
ims_install_hlq: IMS.V15R4M0                 # IMS installation HLQ

# --- RDS Configuration Variables ---
EQARMTD_PATH: "{{ eqa_hlq }}.SEQASAMP(EQARMTD)"
EQARMTSU_PATH: JCLS/EQARMTSU.jcl
EQARMTD_LOCAL_PATH: JCLS/EQARMTD.jcl
eqarmtd_local: files/eqarmtd.env
eqarmtd_dest: /etc/debug/eqarmtd.env
EQARMTD: SYS1.PROCLIB(EQARMTD)

# --- Keystore Configuration ---
keystore: true                                # Enable/disable SSL keystore creation

```

2. Inventory Configuration

inventories/inventory.yml

```

source_system:
  hosts:
    zos_host:
      ansible_host: hostaddress # z/OS hostname or IP
      ansible_user: ibmuser     # SSH username
      ansible_port: ###22       # SSH port

```

Parameter Details:

1. **ansible_host**: Replace with your z/OS system hostname or IP address
2. **ansible_user**: Replace with your z/OS username

3. **ansible_port**: Replace with your SSH port (default is 22)

inventories/host_vars/zos_host.yml

```
---
# Python and ZOAU Paths
PYZ: < path_to_python_installation_on_zos_target > # Python installation path
ZOAU: < path_to_zoau_installation_on_zos_target > # ZOAU installation path

# z/OSMF Configuration
zosmf_host: hostaddress
zosmf_port: ###22
zosmf_user: IBMUSER
zosmf_pass: your_password_here          # ⚠ Change this or use Ansible Vault!

# Ansible Python Interpreter
ansible_python_interpreter: "{{ PYZ }}/bin/python3"

# Privilege Escalation
ansible_become_method: su
ansible_become_password: "{{ zosmf_pass }}"
ansible_become_user: < UID with 0 >
ansible_su_prompt_l10n: FSUM5019 Enter the password for {{
ansible_become_user }}
```

⚠ **Security Note:** Never commit passwords to Git. Use Ansible Vault:

```
# Encrypt the password
ansible-vault encrypt_string 'your_password' --name 'zosmf_pass'
# Use the encrypted value in your file
```

```
# Validate configuration
ansible all -i inventories/inventory.yml -m ping
```

Installation – Automation Execution

There are a lot of steps in installing a debugger and below is a summary of the major implementations in this entire process

- Cleanup - Removes any existing installation (if running Cleanup.yml)
- SMP/E Setup - Extracts and installs base product via SMP/E
- PTF Application - Applies latest fixes and updates
- Product Registration - Registers debugger in PARMLIB
- SVC Installation - Installs and activates debugger SVC
- Server Configuration - Sets up DPS and RDS servers

- Subsystem Integration - Configures CICS/IMS debugging (if selected)
- Security Setup - Creates RACF profiles and permissions
- Verification - Runs IVP (Installation Verification Program)

It can be achieved in either of the below described two approaches.

Option 1: Complete Automation (Recommended)

This way of execution will install and configure the debugger in one shot by running all the playbooks sequentially with just executing one .yaml file.

Step 0: Initialize (Optional - Root Access)

```
ansible-playbook initialise.yaml -i inventories/inventory.yaml -vvv
```

When to use: Only if you need root access for certain operations.

Expected Output:

```
PLAY RECAP *****
***
zos_host                : ok=3    changed=1    unreachable=0    failed=0
```

Step 1: Run all the playbooks at once sequentially.

```
ansible-playbook site.yaml -i inventories/inventory.yaml -vvv
```

Duration: 30-45 minutes

Note: There will be a one-time user input intervention when running EQAWZFS step with a yes/no to proceed the automation further.

Option 2: Step-by-Step Installation

This way of execution gives the user the privilege to run one after the other by checking each step and when required the steps can be performed or resumed self-reliantly.

Run individual playbooks for more control. See detailed steps below.

Step 0: Initialize (Optional - Root Access)

```
ansible-playbook initialise.yaml -i inventories/inventory.yaml -vvv
```

When to use: Only if you need root access for certain operations.

Expected Output:

```
PLAY RECAP *****
***
zos_host                : ok=3    changed=1    unreachable=0    failed=0
```

Step 1: Cleanup

ansible-playbook Cleanup.yml -i inventories/inventory.yml -vvv

Purpose: Removes existing installation for clean install.

Expected Output:

```
PLAY RECAP *****
***
zos_host                : ok=15   changed=12   unreachable=0   failed=0
```

Step 2: SMP/E Installation

ansible-playbook SMPInstall.yml -i inventories/inventory.yml -vvv

Purpose: Installs base product via SMP/E.

Duration: 10-15 minutes

What happens: 1. Uploads HADRXXX.pax.Z to z/OS 2. Extracts PAX file 3. Runs UNZIP job 4. Creates SMP/E environment 5. Receives and applies RELFILES

Expected Output:

```
TASK [Display job submission results]
ok: [zos_host] => {
  "msg": "Job Submission Result for EQAWSMPE : JOB12345"
}

PLAY RECAP *****
***
zos_host                : ok=25   changed=18   unreachable=0   failed=0
```

Verification:

```
# On z/OS
TSO LISTDS 'EQAW.VHR0M0.SEQAMOD'
TSO LISTDS 'EQAW.VHR0M0.SEQASAMP'
```

Step 3: PTF Installation

ansible-playbook PTFInstall.yml -i inventories/inventory.yml -vvv

Purpose: Applies latest PTF.

Duration: 5-10 minutes

Expected Output:

```
PLAY RECAP *****
***
zos_host                : ok=8     changed=5    unreachable=0    failed=0
```


Step 4: Product Configuration

```
ansible-playbook Configuration.yml -i inventories/inventory.yml -vvv
```

Purpose: Registers product and installs SVC.

Duration: 5-10 minutes

What happens: 1. Copies IFAPRDxx to PARMLIB 2. Activates product registration 3. APF-authorizes libraries 4. Installs debugger SVC 5. Runs IVP

Expected Output:

```
TASK [Install SVC]
changed: [zos_host]
```

```
PLAY RECAP *****
***
zos_host                : ok=12   changed=8   unreachable=0   failed=0
```

Verification:

```
# On z/OS - Check SVC is installed
D PROG,SVC
# Look for debugger SVC (usually SVC 245)
```

Step 5: DPS Configuration

```
ansible-playbook DPSConfig.yml -i inventories/inventory.yml -vvv
```

Purpose: Configures Debug Profile Server.

Duration: 5-10 minutes

What happens: 1. Creates STCEQA user 2. Defines RACF resources 3. Copies EQAPROF procedure 4. Customizes config files 5. Starts EQAPROF server

Expected Output:

```
PLAY RECAP *****
***
zos_host                : ok=18   changed=12   unreachable=0   failed=0
```

Verification:

```
# On z/OS
D A,EQAPROF
# Expected: EQAPROF  STARTED  A=0001

netstat -a | grep ###00
# Expected: tcp ... *:###00 ... LISTEN
```

Step 6: CICS Configuration (Optional)

```
ansible-playbook CICSConfig.yml -i inventories/inventory.yml -vvv
```

Purpose: Configures CICS debugging.

Duration: 5 minutes

Post-Configuration: Add datasets to CICS DFHRPL: - IBMUSER.EQAIVP.LOADPDSE - EQAW.VHR0M0.SEQAMOD - SYS1.MIGLIB - SYS1.SIEAMIGE

Step 7: Keystore Creation (Optional)

```
ansible-playbook KeystoreCreation.yml -i inventories/inventory.yml -vvv
```

Purpose: Creates SSL/TLS certificates.

Duration: 3-5 minutes

Step 8: IMS Configuration (Optional)

```
ansible-playbook IMSConfig.yml -i inventories/inventory.yml -vvv
```

Purpose: Configures IMS debugging.

Duration: 5-10 minutes

Step 9: RDS Configuration

```
ansible-playbook RDSConfig.yml -i inventories/inventory.yml -vvv
```

Purpose: Configures Remote Debug Server.

Duration: 5 minutes

Verification:

```
# On z/OS
D A,EQARMTD
# Expected: EQARMTD  STARTED  A=0002
```

Specific Use Case Scenarios

Scenario 1: Recovery - Reinstallation

Objective: Quickly restore debugger after system failure or migration.

Prerequisites: - Backup of variables.yml and inventory files - Access to installation files - Clean z/OS system or cleaned environment

Recovery Steps:

```
# 1. Restore configuration files from backup
cp backup/variables.yml .
```

```
cp backup/inventories/inventory.yml inventories/
```

```
# 2. Run complete automation
```

```
ansible-playbook site.yml -i inventories/inventory.yml -vvv
```

```
# 3. Verify all services
```

```
ssh user@zos
```

```
D A,EQAPROF
```

```
D A,EQARMTD
```

```
# 4. Test connectivity
```

```
# Connect from IDE to verify
```

Recovery Time: 30-45 minutes (automated)

Manual Recovery Time: 4-6 hours (for comparison)

Scenario 2: Upgrade - Applying New PTF

Objective: Apply a new PTF to existing installation without full reinstall.

Configuration:

```
# Update variables.yml with new PTF
```

```
ptf_file: IBM.HADRXXX.PTF#### # New PTF number
```

Upgrade Steps:

```
# 1. Download new PTF to files/ directory
```

```
cp IBM.HADRXXX.PTF#### files/
```

```
# 2. Update variables.yml with new PTF name
```

```
vi variables.yml
```

```
# Change: ptf_file: IBM.HADRXXX.PTF####
```

```
# 3. Stop debug servers
```

```
ssh user@zos
```

```
P EQAPROF
```

```
P EQARMTD
```

```
# 4. Apply new PTF
```

```
ansible-playbook PTFInstall.yml -i inventories/inventory.yml -vvv
```

```
# 5. Restart servers
```

```
S EQAPROF
```

```
S EQARMTD
```

```
# 6. Verify
```

```
D A,EQAPROF
```

```
D A,EQARMTD
```

Downtime: 10-15 minutes

Scenario 3: Multi-Region CICS Setup

Objective: Configure debugging for multiple CICS regions.

Approach: Run CICSConfig.yml multiple times with different variables.

Configuration for Region 1:

```
# variables.yml - CICS01
CSDPRFX: CICS61.CICS01
cics_region: CICS01
dtcn_port: ###01
port_internal: ###01
port_external_secure: ###02
```

Configuration for Region 2:

```
# variables.yml - CICS02
CSDPRFX: CICS61.CICS02
cics_region: CICS02
dtcn_port: ###11
port_internal: ###11
port_external_secure: ###12
```

Installation Steps:

```
# 1. Configure first CICS region
ansible-playbook CICSConfig.yml -i inventories/inventory.yml -vvv

# 2. Update variables.yml for second region
vi variables.yml
# Update CSDPRFX, cics_region, and ports

# 3. Configure second CICS region
ansible-playbook CICSConfig.yml -i inventories/inventory.yml -vvv

# Repeat for additional regions
```

Scenario 4: Rollback to Previous Version

Objective: Rollback to previous installation if issues occur.

Steps:

```
# 1. Run cleanup to remove current installation
ansible-playbook Cleanup.yml -i inventories/inventory.yml -vvv

# 2. Restore previous configuration
cp backup/variables.yml.previous variables.yml
```

3. Use previous PTF file

```
cp backup/files/IBM.HADRXXX.PTF####files/
```

4. Reinstall

```
ansible-playbook site.yml -i inventories/inventory.yml -vvv
```

Post-Installation Verification

Verification Checklist

After installation, verify the following:

1. Dataset Verification

On z/OS, check datasets exist

```
TSO LISTDS 'EQAW.VHR0M0.*'
```

Expected Datasets: - EQAW.VHR0M0.SEQAMOD - Load modules - EQAW.VHR0M0.SEQASAMP - Sample programs - EQAW.VHR0M0.SEQAAUTH - Authorized modules - EQAW.VHR0M0.SEQABMOD - Base modules

2. zFS Directory Verification

On z/OS

```
ls -la /u/ibmuser/usr/lpp/IBM/debugger
```

```
ls -la /u/ibmuser/etc/debug
```

```
ls -la /u/ibmuser/var/debug
```

Expected Directories:

```
/u/ibmuser/usr/lpp/IBM/debugger/
```

```
|— bin/
|— lib/
|— samples/
```

```
/u/ibmuser/etc/debug/
```

```
|— eqaprof.env
|— server.xml
|— dtcn.ports
```

```
/u/ibmuser/var/debug/
```

```
|— logs/
```

3. Started Task Verification

On z/OS

```
D A,EQAPROF
```

```
D A,EQARMTD
```

Expected Output:

```
EQAPROF  STARTED  A=0001
EQARMTD  STARTED  A=0002
```

4. Port Verification

```
# On z/OS
netstat -a | grep -E "####00|####01|####02|####03|####04"
```

Expected Output:

tcp	0	0 *:####00	*:*	LISTEN
tcp	0	0 *:####01	*:*	LISTEN
tcp	0	0 *:####02	*:*	LISTEN
tcp	0	0 *:####03	*:*	LISTEN
tcp	0	0 *:####04	*:*	LISTEN

5. SVC Verification

```
# On z/OS
D PROG,SVC
```

Expected Output:

SVC 245 EQASVC (or similar SVC number)

6. APF Authorization Verification

```
# On z/OS
D PROG,APF
```

Expected Output:

```
EQAW.VHR0M0.SEQAAUTH,VOL=T#####
EQAW.VHR0M0.SEQABMOD,VOL=T#####
EQAW.VHR0M0.SEQAMOD,VOL=T#####
```

7. RACF Security Verification

```
# On z/OS
RLIST FACILITY EQA.* ALL
RLIST STARTED EQAPROF.* ALL
```

Expected Output:

```
FACILITY: EQA.PROFILE.SERVER
STARTED: EQAPROF
```

Functional Testing

Test 1: Batch Program Debugging

```
# 1. Submit test job
# 2. Connect debugger
# 3. Set breakpoint
# 4. Step through code
# 5. Inspect variables
```

Expected Result: Debugger connects and allows source-level debugging.

Test 2: CICS Transaction Debugging

```
# 1. Start DTCN transaction in CICS
# 2. Create debug profile
# 3. Run transaction
# 4. Debug session starts
```

Expected Result: DTCN intercepts transaction and starts debug session.

Test 3: IMS Program Debugging

```
# 1. Configure IMS debug profile
# 2. Run IMS transaction
# 3. Debugger intercepts
```

Expected Result: IMS program debugging works as expected.

Test 4: Remote Debugging from IDE

```
# 1. Configure IDE connection
#   Host: zos_hostname
#   Port: ###00
# 2. Connect from IDE
# 3. Set breakpoints
# 4. Run program
```

Expected Result: IDE connects and debugging works.

Performance Verification

Check Server Response Time

```
# Test Liberty server response
curl -k https://zos_hostname:###00/health
```

```
# Expected response time: < 2 seconds
```

Check Log Files

```
# On z/OS
tail -f /u/ibmuser/var/debug/logs/eqaprof.log
tail -f /u/ibmuser/var/debug/logs/liberty.log
```

Look for: - No error messages - Successful startup messages - Normal operation logs

Troubleshooting Failed Verification

If verification fails, check:

1. **Ansible logs:** `cat ansible.log | grep -i error`
2. **Job outputs:** Review JCL job outputs on z/OS
3. **System logs:** Check z/OS SYSLOG for errors
4. **Permissions:** Verify user has required access

5. **Network:** Verify ports are accessible

Troubleshooting

Common Issues and Solutions

Issue 1: SSH Connection Failed

Symptom:

```
fatal: [zos_host]: UNREACHABLE! => {"changed": false, "msg": "Failed to connect to the host via ssh"}
```

Possible Causes: - SSH key not copied to z/OS - Wrong hostname/IP in inventory - Wrong SSH port - Firewall blocking connection

Solutions:

1. Verify SSH connectivity manually

```
ssh -p ###22 ibmuser@hostaddress
```

2. Re-copy SSH key

```
ssh-copy-id -p ###22 ibmuser@hostaddress
```

3. Check inventory.yml settings

```
cat inventories/inventory.yml
```

4. Test with verbose SSH

```
ssh -vvv -p ###22 ibmuser@hostaddress
```

Issue 2: Job Failed with RC > 0

Symptom:

```
TASK [Step 1 - Unzip - UNZIP] *****  
***
```

```
fatal: [zos_host]: FAILED! => {"changed": false, "msg": "Job failed with RC 12"}
```

Solutions:

1. Check job output on z/OS

Use SDSF or z/OSMF to view job output

2. Review JCL for errors

```
cat JCLS/UNZIP.jcl
```

3. Check dataset allocations

Verify volumes have space

4. Check for dataset conflicts
TSO LISTDS 'dataset.name'

Issue 3: EQAPROF Won't Start

Symptom:

D A,EQAPROF
IEE341I EQAPROF NOT FOUND

Solutions:

1. Check PROCLIB member exists
TSO LISTDS 'USER.PROCLIB(EQAPROF)'

2. Verify PROCLIB is in JES2 concatenation
D PROCLIB

3. Check for errors in procedure
cat JCLS/EQAPROF.jcl

4. Manually start with full path
S EQAPROF,PROCLIB=USER.PROCLIB

5. Check system log for errors
Review SYSLOG for startup errors

Issue 4: Port Already in Use

Symptom:

Port ###00 is already in use

Solutions:

1. Check what's using the port
netstat -a | grep ###00

2. Stop conflicting process
P <jobname>

3. Change port in variables.yml
vi variables.yml
Update liberty_port to different value

4. Rerun DPS configuration
ansible-playbook DPSConfig.yml -i inventories/inventory.yml -vvv

Issue 5: APF Authorization Failed

Symptom:

SETPROG APF command failed

Solutions:

*# 1. Verify user has authority
Need OPERATIONS or equivalent*

2. Check dataset exists
TSO LISTDS 'EQAW.VHR0M0.SEQAAUTH'

*# 3. Verify volume
Dataset must be on correct volume*

4. Manual APF authorization
SETPROG APF,ADD,DSN=EQAW.VHR0M0.SEQAAUTH,VOL=T#####

*# 5. Make permanent in PROGxx
Add to SYS1.PARMLIB(PROGxx)*

Issue 6: RACF Permission Denied

Symptom:

ICH408I USER(IBMUSER) NOT AUTHORIZED TO FACILITY CLASS

Solutions:

1. Check RACF permissions
RLIST FACILITY EQA.* ALL

2. Grant access
PERMIT EQA.PROFILE.SERVER CLASS(FACILITY) ID(IBMUSER) ACCESS(READ)
SETROPTS RACLIST(FACILITY) REFRESH

3. Verify access
RLIST FACILITY EQA.PROFILE.SERVER AUTHUSER

4. Check started task permissions
RLIST STARTED EQAPROF.* ALL

Issue 7: Java/Liberty Not Found

Symptom:

Java or Liberty directory not found

Solutions:

1. Verify Java installation
ls -la /usr/lpp/java/

```
# 2. Verify Liberty installation
ls -la /usr/lpp/liberty_zos/

# 3. Update paths in host_vars
vi inventories/host_vars/zos_host.yml
# Update JAVADIR and Liberty_dir

# 4. Test Java
/usr/lpp/java/J17.0_64/bin/java -version
```

Issue 8: Dataset Space Issues

Symptom:

B37-04 SPACE ERROR

Solutions:

```
# 1. Check volume space
D U,VOL=T#####

# 2. Free up space or use different volume
# Update variables.yml with volume that has space

# 3. Increase space allocations in JCL
# Edit JCLS/*.jcl files

# 4. Clean up old datasets
TSO DELETE 'old.dataset.**'
```

Issue 9: SVC Installation Failed

Symptom:

SVC 245 already in use

Solutions:

```
# 1. Check current SVC usage
D PROG,SVC

# 2. Use different SVC number
# Edit EQAWISVC.jcl to use different SVC

# 3. Remove old SVC if safe
# Consult system programmer

# 4. Verify SVC is debugger SVC
# Check if it's from previous installation
```

Issue 10: Debugger Won't Connect from IDE

Symptom:

Connection refused or timeout

Solutions:

1. Verify EQAPROF is running
D A,EQAPROF

2. Check port is listening
netstat -a | grep ###00

3. Test connectivity from client
telnet zos_hostname ###00

4. Check firewall rules
Ensure port ###00 is open

5. Verify Liberty server is up
tail -f /u/ibmuser/var/debug/logs/liberty.log

6. Check SSL configuration if using HTTPS
Verify certificates are valid

Diagnostic Commands

Collect Diagnostic Information

On z/OS - Run these commands and save output

1. System information
D IPLINFO
D SYMBOLS

2. Active jobs
D A,ALL

3. APF List
D PROG,APF

4. SVC List
D PROG,SVC

5. Network status
netstat -a

6. Dataset List
TSO LISTDS 'EQAW.VHR0M0.*'

7. Directory contents

```
ls -laR /u/ibmuser/usr/lpp/IBM/debugger
ls -laR /u/ibmuser/etc/debug
ls -laR /u/ibmuser/var/debug
```

8. Log files

```
tail -100 /u/ibmuser/var/debug/logs/eqaprof.log
tail -100 /u/ibmuser/var/debug/logs/liberty.log
```

9. RACF information

```
RLIST FACILITY EQA.* ALL
RLIST STARTED EQAPROF.* ALL
```

Ansible Diagnostic Commands

On control node

1. Verify Ansible version

```
ansible --version
```

2. List collections

```
ansible-galaxy collection list
```

3. Test connectivity

```
ansible all -i inventories/inventory.yml -m ping
```

4. List variables

```
ansible-inventory -i inventories/inventory.yml --host zos_host --yaml
```

5. Syntax check

```
ansible-playbook --syntax-check site.yml -i inventories/inventory.yml
```

6. Dry run

```
ansible-playbook site.yml -i inventories/inventory.yml --check
```

7. Review Logs

```
tail -f ansible.log
grep -i error ansible.log
grep -i failed ansible.log
```

Getting Help

If you cannot resolve the issue:

1. Review Documentation:

- IBM z/OS Debugger documentation- [Link](#)

2. Check Logs:

- ansible.log - Ansible execution log

- z/OS SYSLOG - System messages
 - /u/ibmuser/var/debug/logs/ - Debug server logs
- 3. Collect Information:**
- Run diagnostic commands in Diagnostic commands section of this document.
 - Save job outputs
 - Note error messages
- 4. Contact Support:**
- Provide collected diagnostic information
 - Include Ansible version and collection versions
 - Describe steps taken and results

Appendices

NOTE: ### is used in Port numbers to depict the equivalent ports assigned on that machine of choice to run this automation.

Appendix A: Variable Reference Table

Variable	Type	Default	Description	Required
smphlq	String	IBMUSER.HADR H00	SMP/E dataset HLQ	Yes
volser	String	T#####	SMP/E control volume	Yes
tvol	String	T#####	Target library volume	Yes
dvol	String	T#####	Distribution volume	Yes
eqa_hlq	String	EQAW.VHR0M0	Debugger dataset HLQ	Yes
eqa_path_prefi x	String	/u/ibmuser/	zFS base path	Yes
remote_path_re lfiles	String	/u/ibmuser/S MP/relfiles	Temp RELFILES path	Yes
ptf_file	String	IBM.HADRH00. U007312	PTF filename	Yes
PARMLIB	String	USER.PARMLIB	PARMLIB dataset	Yes
IFAPRDxx	String	TZ	IFAPRD suffix	Yes
registration	String	EQAWIFAT	Registration member	Yes
ivp_LNGPRFX	String	IGY.V6R4M0	COBOL compiler HLQ	Yes
ivp_LIBPRFX	String	CEE	LE library HLQ	Yes
PROCLIB	String	USER.PROCLIB	Started task PROCLIB	Yes

Variable	Type	Default	Description	Required
DFHHLQ	String	DFH.V6R3M0.CICS	CICS HLQ	If using CICS
CFGDIR	String	/u/ibmuser/etc/debug	Config directory	Yes
WRKDIR	String	/u/ibmuser/var/debug	Work directory	Yes
JAVADIR	String	/usr/lpp/java/J17.0_64	Java path	Yes
liberty_dir	String	/usr/lpp/liberty_zos/current	Liberty path	Yes
liberty_port	Integer	####00	Liberty port	Yes
CSDPRFX	String	CICS61.CICS01	CICS CSD prefix	If using CICS
dtn_port	Integer	####01	DTCN port	If using CICS
cics_region	String	CICS01	CICS region name	If using CICS
port_internal	Integer	####01	Internal debug port	If using CICS
port_external_secure	Integer	####02	Secure debug port	If using CICS
ims_region	String	IMSREG1	IMS region name	If using IMS
ims_debugger_port	Integer	####03	IMS debug port	If using IMS
ims_dtn_port	Integer	####04	IMS DTCN port	If using IMS
ims_install_hlq	String	IMS.V15R4M0q	IMS HLQ	If using IMS
keystore	Boolean	true	Enable SSL keystore	No

Appendix B: Port Reference (With respect to H3N or equivalent systems)

Port	Service	Protocol	Purpose
22	SSH	TCP	Ansible connectivity
####00	Liberty	TCP/HTTPS	Debug Profile Server
####01	DTCN	TCP	CICS DTCN transaction
####02	Secure Debug	TCP/HTTPS	CICS secure debugging
####03	IMS Debug	TCP	IMS debug server
####04	IMS DTCN	TCP	IMS DTCN transaction

Appendix C: Dataset Naming Conventions

Dataset Pattern	Purpose	Example
<smphlq>.CSI	SMP/E CSI	IBMUSER.HADRH00.CSI

Dataset Pattern	Purpose	Example
<smphlq>.SMPLOG	SMP/E log	IBMUSER.HADRH00.SMPLOG
<eqa_hlq>.SEQAMOD	Load modules	EQAW.VHR0M0.SEQAMOD
<eqa_hlq>.SEQASAMP	Samples	EQAW.VHR0M0.SEQASAMP
<eqa_hlq>.SEQAAUTH	Authorized modules	EQAW.VHR0M0.SEQAAUTH
<user>.IMS.CONFIG	IMS config	IBMUSER.IMS.CONFIG
<user>.IMS.PROFILE	IMS profiles	IBMUSER.IMS.PROFILE

Appendix D: JCL Templates

The automation uses the following JCL templates in the JCLS/ directory:

JCL File	Purpose	Used By
UNZIP.jcl	Extract RELFILES	SMPInstall.yml
EQAWSMP.jcl	SMP/E setup	SMPInstall.yml
RECEIVE.jcl	Receive PTF	PTFInstall.yml
APPLY.jcl	Apply PTF	PTFInstall.yml
ACCEPT.jcl	Accept PTF	PTFInstall.yml
EQAWISVC.jcl	Install SVC	Configuration.yml
EQAPROF.jcl	DPS procedure	DPSCConfig.yml
EQAPRFSU.jcl	DPS setup	DPSCConfig.yml
EQAWCCSD.jcl	CICS CSD update	CICSCConfig.yml
EQAWIVCT.jcl	CICS IVP	CICSCConfig.yml
EQARMTSU.jcl	RDS setup	RDSCConfig.yml
EQARMTD.jcl	RDS procedure	RDSCConfig.yml

Appendix E: Security Resources

RACF Profiles Created

Profile	Class	Purpose
EQA.PROFILE.SERVER	FACILITY	DPS server access
EQA.CICS.<region>	FACILITY	CICS debugging
EQA.IMS.<region>	FACILITY	IMS debugging
EQAPROF	STARTED	DPS started task
EQARMTD	STARTED	RDS started task
BPX.SRV.<user>	SURROGAT	Surrogate permissions

Required User Permissions

Permission	Purpose
TSO access	Submit jobs, run commands
Dataset CREATE	Create installation datasets
Dataset DELETE	Remove old datasets
RACF SPECIAL	Define security resources
OPERATIONS	APF authorization, SVC install

Appendix F: File Structure

```
Debug-Config-Automation-Bob-IMS-Config/
├── ansible.cfg                # Ansible configuration
├── ansible.log                # Execution log
├── AUTOMATION_GUIDE.md       # This guide
├── README.md                  # Quick start guide
├── IMS_README.md              # IMS-specific guide
├── variables.yml              # Main configuration
├── site.yml                   # Master playbook
├── initialise.yaml            # Root surrogate setup
├── Cleanup.yml                # Cleanup playbook
├── SMPInstall.yml             # SMP/E installation
├── PTFInstall.yml             # PTF installation
├── Configuration.yml          # Product configuration
├── DPSConfig.yml              # DPS configuration
├── CICSConfig.yml             # CICS configuration
├── IMSConfig.yml              # IMS configuration
├── RDSConfig.yml              # RDS configuration
├── KeystoreCreation.yml        # SSL keystore creation
├── files/                     # Installation files
│   ├── HADRXXX.pax.Z          # Base product
│   ├── HADRHO0.readme.txt     # Release notes
│   └── IBM.HADRXXX.PTF####    # PTF file
├── inventories/               # Inventory configuration
│   ├── inventory.yml          # Host inventory
│   ├── group_vars/
│   │   └── all.yml            # Global variables
│   └── host_vars/
│       └── zos_host.yml        # Host-specific variables
├── JCLS/                      # JCL templates
│   ├── UNZIP.jcl
│   ├── EQAWSMPE.jcl
│   ├── RECEIVE.jcl
│   ├── APPLY.jcl
│   ├── ACCEPT.jcl
│   ├── EQAWISVC.jcl
│   ├── EQAPROF.jcl
│   ├── EQAPRFSU.jcl
│   └── EQAWCCSD.jcl
```

```
| EQAWIVCT.jcl
| EQARMTSU.jcl
| EQARMTD.jcl
```

Appendix G: Related Documentation

- **IBM z/OS Debugger Documentation:** <https://www.ibm.com/docs/en/developer-for-zos>
- **Ansible Documentation:** <https://docs.ansible.com/>
- **IBM z/OS Core Collection:** https://galaxy.ansible.com/ibm/ibm_zos_core

Appendix H: Quick Reference Commands

Installation

```
ansible-playbook site.yml -i inventories/inventory.yml -vvv
```

Individual playbooks

```
ansible-playbook SMPInstall.yml -i inventories/inventory.yml -vvv
ansible-playbook PTFInstall.yml -i inventories/inventory.yml -vvv
ansible-playbook Configuration.yml -i inventories/inventory.yml -vvv
ansible-playbook DPSConfig.yml -i inventories/inventory.yml -vvv
ansible-playbook CICSCfg.yml -i inventories/inventory.yml -vvv
ansible-playbook IMSConfig.yml -i inventories/inventory.yml -vvv
ansible-playbook RDSCfg.yml -i inventories/inventory.yml -vvv
```

Cleanup

```
ansible-playbook Cleanup.yml -i inventories/inventory.yml -vvv
```

Verification

```
ansible all -i inventories/inventory.yml -m ping
ansible-playbook --syntax-check site.yml -i inventories/inventory.yml
```

On z/OS

```
D A,EQAPROF
D A,EQARMTD
D PROG,SVC
D PROG,APF
netstat -a | grep ###00
```

```
# Check DPS server
# Check RDS server
# Check SVC
# Check APF List
# Check ports
```

Conclusion

This automation framework provides a comprehensive solution for installing and configuring IBM z/OS Debugger. By following this guide, you can:

- Install the debugger in 30-45 minutes (vs. 4-6 hours manually)
- Ensure consistent configuration across environments
- Easily replicate installations

- Quickly recover from failures
 - Apply updates with minimal downtime
-